

Cognizant Digital Nurture 5.0

Angular Track

Week 1 Learning Progress Report

Candidate Name: Pooja V

College: St. Joseph's Institute of Technology

Department: Electrical and Electronics Engineering (EEE)

Program: Cognizant Digital Nurture 5.0

Track: Java FSE - Angular

Week: Week 1

Topics Covered:

- Design Patterns and Principles
- Data Structures and Algorithms
- PL/SQL Programming

Objective

The objective of Week 1 is to build a strong foundation in software engineering concepts, programming principles, problem-solving techniques, database programming, and testing methodologies. These concepts serve as the basis for advanced topics such as Spring Boot, REST APIs, Hibernate, and Angular in the upcoming weeks.

Prepared By

Pooja V

Cognizant Digital Nurture 5.0 - Deep Skilling Program

Design Patterns and Principles

Exercise 1 - Singleton Pattern

```
package logger;

public class Logger {

    private static Logger instance;

    private Logger() {
        System.out.println("Logger Instance Created");
    }

    public static Logger getInstance() {

        if (instance == null) {
            instance = new Logger();
        }

        return instance;
    }

    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}

package logger;

public class SingletonTest {

    public static void main(String[] args) {

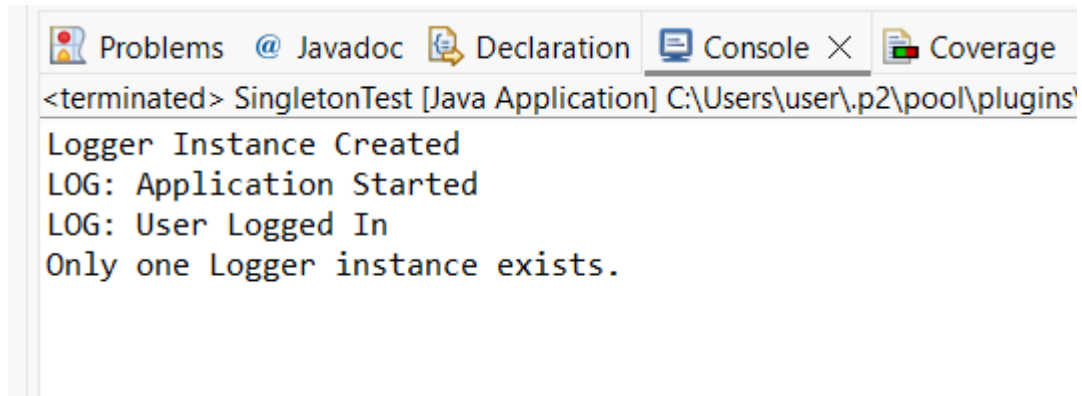
        Logger logger1 = Logger.getInstance();
```

```
Logger logger2 = Logger.getInstance();

logger1.log("Application Started");
logger2.log("User Logged In");

System.out.println("Logger1: " + logger1);
System.out.println("Logger2: " + logger2);

if (logger1 == logger2) {
    System.out.println("Only one Logger instance exists.");
}
```



Exercise 2 - FactoryMethodTest

```
interface Document {  
    void open();  
}  
  
class WordDocument implements Document {  
    public void open() {  
        System.out.println("Opening Word Document");  
    }  
}  
  
class PdfDocument implements Document {  
  
    @Override  
    public void open() {  
        System.out.println("Opening PDF Document");  
    }  
}  
  
class ExcelDocument implements Document {  
  
    @Override  
    public void open() {  
        System.out.println("Opening Excel Document");  
    }  
}  
  
abstract class DocumentFactory {  
  
    public abstract Document createDocument();  
}  
  
class WordDocumentFactory extends DocumentFactory {  
  
    @Override  
    public Document createDocument() {  
        return new WordDocument();  
    }  
}  
  
class PdfDocumentFactory extends DocumentFactory {
```

```

@Override
public Document createDocument() {
    return new PdfDocument();
}
}

class ExcelDocumentFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new ExcelDocument();
    }
}

public class FactoryMethodTest {

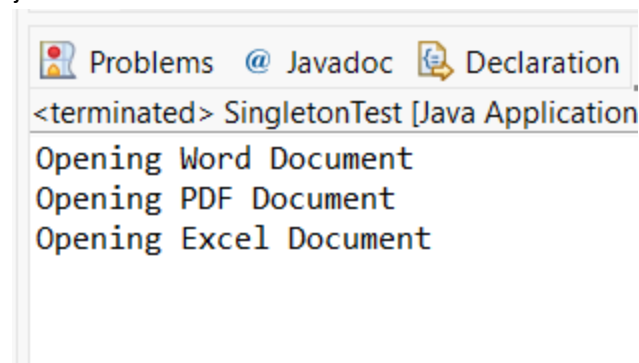
    public static void main(String[] args) {

        DocumentFactory wordFactory = new WordDocumentFactory();
        DocumentFactory pdfFactory = new PdfDocumentFactory();
        DocumentFactory excelFactory = new ExcelDocumentFactory();

        Document word = wordFactory.createDocument();
        Document pdf = pdfFactory.createDocument();
        Document excel = excelFactory.createDocument();

        word.open();
        pdf.open();
        excel.open();
    }
}

```



Algorithms_Data Structures

Exercise 3 – SearchTest

```
package ecommerce;

public class SearchTest {

    public static Product linearSearch(Product[] products, int id) {
        for (Product p : products) {
            if (p.productId == id) {
                return p;
            }
        }

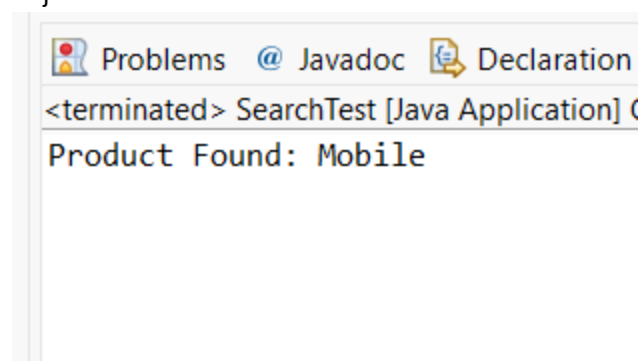
        return null;
    }

    public static void main(String[] args) {

        Product[] products = {
            new Product(101, "Laptop", "Electronics"),
            new Product(102, "Mobile", "Electronics"),
            new Product(103, "Shoes", "Fashion")
        };

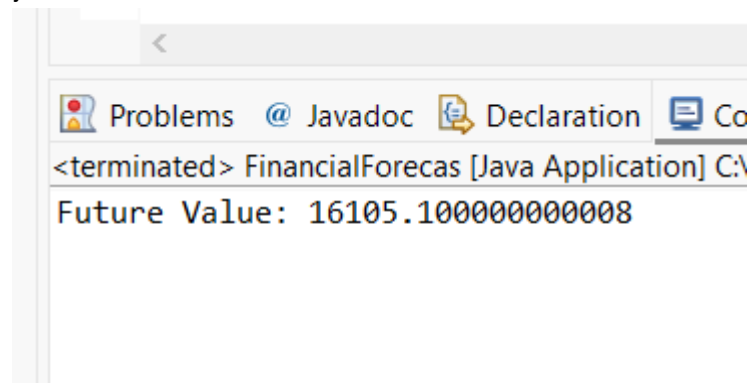
        Product result = linearSearch(products, 102);

        if (result != null) {
            System.out.println("Product Found: " + result.productName);
        } else {
            System.out.println("Product Not Found");
        }
    }
}
```



Exercise 4 - Financial Forecasting

```
public class FinancialForecast {  
  
    public static double futureValue(double amount, double rate, int years) {  
  
        if (years == 0) {  
            return amount;  
        }  
  
        return futureValue(amount, rate, years - 1) * (1 + rate);  
    }  
  
    public static void main(String[] args) {  
  
        double presentValue = 10000;  
        double growthRate = 0.10;  
        int years = 5;  
  
        double result = futureValue(presentValue, growthRate, years);  
  
        System.out.println("Future Value: " + result);  
    }  
}
```



PL/SQL programming

Exercise 5 – Control Structures

```
DECLARE
    age NUMBER := 65;
    discount NUMBER;
BEGIN

    IF age >= 60 THEN
        discount := 20;
    ELSIF age >= 40 THEN
        discount := 10;
    ELSE
        discount := 5;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Discount = ' || discount || '%');

END;
```

```
Output:
Discount = 20%
```


Exercise 6 – Control Structures

```
CREATE TABLE employees ( emp_id NUMBER PRIMARY KEY, emp_name VARCHAR2(50),  
salary NUMBER );
```

```
INSERT INTO employees VALUES (101,'POOJA',30000);
```

```
INSERT INTO employees VALUES (102,'KIRAN',40000);
```

```
COMMIT; CREATE OR REPLACE PROCEDURE update_salary( p_emp_id IN NUMBER,  
p_increment IN NUMBER )
```

```
IS BEGIN UPDATE employees SET salary = salary + p_increment WHERE emp_id = p_emp_id;  
COMMIT;
```

```
END;
```

```
/ BEGIN update_salary(101,5000);
```

```
END;
```

```
/ SELECT * FROM employees;
```

Output:

EMP_ID	EMP_NAME	SALARY
101	Pooja	35000
102	Kiran	40000